马斯克说到做到：开源X平台核心推荐算法

原创 腾讯程序员 腾讯技术工程 Feb 12, 2026, 1:39 AM 广东 548 listened



作者：腾讯广告推荐技术团队，周超博士

> 1，X开源内容推荐Feed on X召回、精排推理、策略代码。参数配置小应是简化版本。
> 2，召回是双塔模型，用户塔输入是【用户特征，S长度的行为序列】，内容塔是帖子标识与帖子作者。用户塔是双层的Causal Attention的Decoder-Only的Transformer，Output取所有位置的平均池化，进行L2归一化；内容塔是两层MLP；采用dot product检索。
> 3，精排采用双层Causal Attention的Decoder-Only的Transformer。精排输入是【用户特征，S长度行为序列，候选1~候选N】，对候选采用Candidate Isolation。

1. 结论

● X开源的是内容推荐(Feed on X)的召回、精排的在线请求视角的推理、策略/过滤代码，未开源训练(含Loss设计、样本组织的代码）。

● 模型参数配置较小（例如用户行为序列是32），应是实际部署代码的简化Demo版本。推测开源核心目的是为了体现X的透明性，解除广告主与X的用户对X的Feed推荐策略受到非内容因素干扰的担忧。

● 召回

○双塔模型，用户塔是双层的Decoder-Only的Transformer，采用Causal Attention，Output是取所有位置的平均池化(mean pooling)，再进行L2的归一化；内容塔是两层的MLP；采用dot product进行检索(因用户、内容都做了L2归一化，等价于cosine)。

○输入：用户塔【用户特征，用户行为序列S长度】，内容塔是帖子标识+帖子作者

● 精排

○ 双层的Decoder-Only的Transformer，对于用户特征、用户行为序列是Causal Attention，对于候选是Candidate Isolation。

○ 输入：【用户特征，用户行为序列S长度，候选1，...，候选N】

● 召回、精排的用户ID、作者ID、帖子ID都用multi-hash查询embedding table。

2.背景

● X(原twitter)的开源项目： **GitHub - xai-org/x-algorithm: Algorithm powering the For You feed on X**

● "For You" feed on X（内容推荐）：



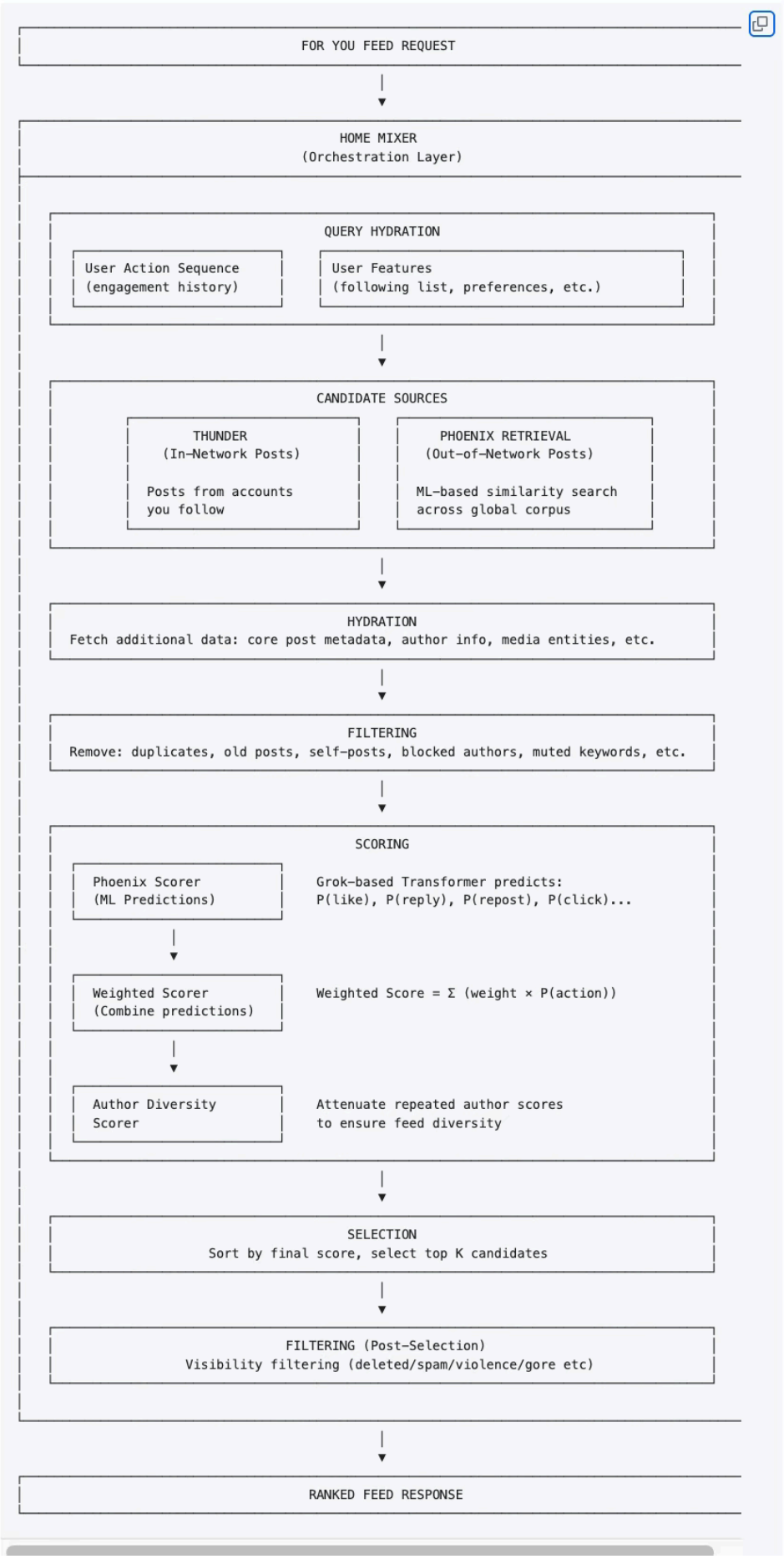
● 只开源了推理视角的模型（召回、精排）、策略/过滤代码，没有开源训练的代码（Loss、样本等）。

● 来自Elon Musk本人的点评： We know the algorithm is dumb and needs massive improvements, but at least you can see us struggle to make it better in real-time and **with transparency**. No other social media companies do this.


Top
Latest
Elon Musk @elonmusk · Jan 20
We know the algorithm is dumb and needs massive improvements, but at least you can see us struggle to make it better in real-time and with transparency.
No other social media companies do this.
Engineering @XEng · Jan 20
We have open-sourced our new 𝕏 algorithm, powered by the same transformer architecture as xAI's Grok model.
Check it out here: github.com/xai-org/x-algo...
8.2K
10K
94K
28M

3.架构

System Architecture



● Home Mixer：整体系统的名字。

- Query Hydration，获取用户的近期的历史行为与Metadata（例如关注列表）：
  - User Action Sequence，用户行为序列
  - User Features，关注列表等
- Candidate Sourcing，从2个候选池拉取帖子
  - Thunder, In-Network Posts，关注的账号的帖子
  - Phoenix Retrieval，Out-of-Network Posts，全局贴子里基于相似度找到的
- Candidate Hydration，为每个候选补充数据【文档里描述，代码里没有使用这些信息】
  - 帖子数据：文本、多媒体
  - 作者信息：用户名称，认证状态
  - 视频时长
  - 订阅状态，区分是否是订阅才可看的帖子
- 前过滤：
  - 去除重复的、老帖子、自己的帖子、屏蔽的作者的帖子、屏蔽的关键词的帖子、过去看过的帖子、不是订阅状态不可见的帖子。
- Scoring
  - **Phoenix Scorer**：
    - Grok-based transformer model，包括两阶段，召回与排序。
    - 19个目标打分，P(like)、P(reply)、P(repost)、P(click)等
  - weighted scorer，多目标分数融合，得到一个最终分数【权重参数未开源】
  - Author diversity Scorer，作者多样性评分。
  - OON Score：对于不在订阅列表的帖子调整分值。
- Selection
  - 基于Final Score，选取Top-K的候选
- 后过滤
  - 重复的PostID
  - Meta信息获取不全的帖子
  - 超过时效性的老帖子

- 用户自己的帖子
- 内容一致的帖子
- 用户没有订阅的帖子
- 用户已经阅读过的帖子【已经阅读的过滤】
- 用户在同Session已经曝光过的帖子【重复曝光过滤】
- 用户屏蔽关键词的帖子
- 用户屏蔽作者的帖子
- 已经被删除、识别为spam、识别为暴力等帖子。
- 对来自同一个twitter下多次交互的conversation进行去重

Thunder：

● 存储于内存里的，记录所有用户的近期的帖子的存储

● 从Kafka里，记录帖子的创建、删除事件。

● 维持每个用户的原创帖子、回复、转发、视频帖子。

● 在“in-network"检索中，提供用户关注的账号列表的获取。

● 自动删除过期的帖子。

其他关键设计：

● Hash-Based Embeddings：使用了多个hash函数来做embedding lookup。应该是为了解决hash冲突。

● No Hand Engineered Features：没有内容相关性的人工设计的特征。

● Candidate Isolation in Ranking：候选之间不能互相attention。

● Multi-Action Prediction：多目标预估，而不是单一的相关性分数。

4. Phoenix-模型整体

● 代码：**x-algorithm/phoenix at main · xai-org/x-algorithm · GitHub**

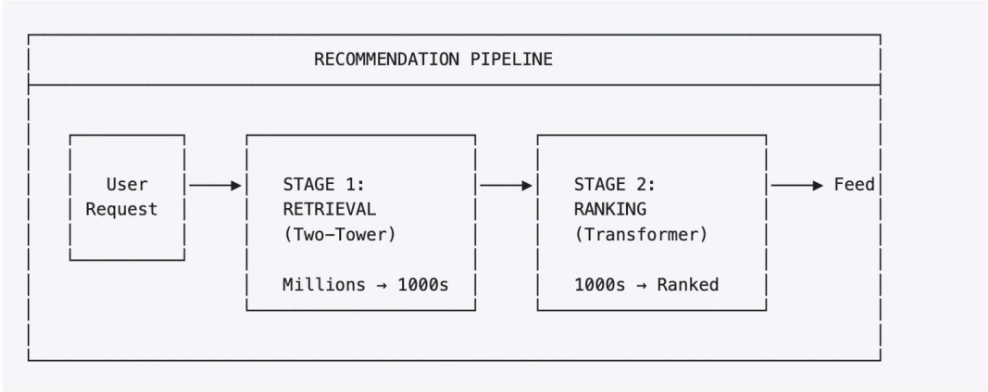
- recsys_retrieval_mdel.py ： Retrieval召回模型
- run_retrieval.py：召回模型运行示例
- test_recsys_retrieval_model.py：召回模型测试
- recsys_model.py ： Ranking排序模型

- run_ranker.py： 排序模型运行示例
- test_recsys_model.py：排序模型测试
- grok.py: Transformer基础架构(基于Grok-1在内容推荐场景改造)
- runners.py： 模型推理逻辑

● 两阶段：召回，排序。

● 应用于Feeds的内容推荐。

Two-Stage Recommendation Pipeline



5. Phoenix-模型-召回

● 双塔模型，应用于从非用户关注的账户里检索潜在感兴趣的帖子，百万级别候选，返回千级别候选

● User Tower：

- 【用户特征，用户历史行为1，用户历史行为2...，用户历史行为S】。
- Decoder-Only Tranformer，异构序列的输入，Causal Self-Attention，每个位置可以看到自己及之前的信息。
- Output是取所有位置的平均池化(mean pooling)，再进行L2的归一化。【非Next Item Prediction】

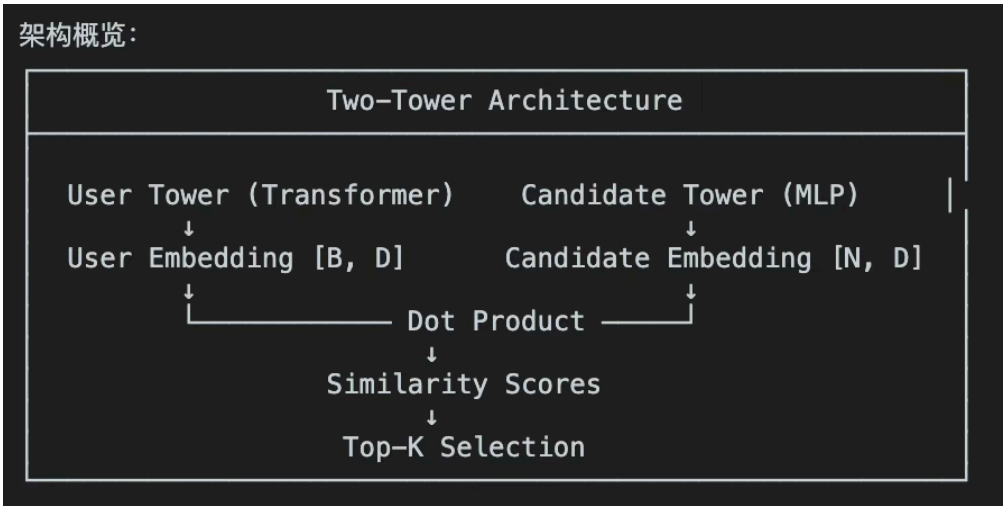
● Candidate Tower：

- 特征是候选帖子唯一标识+候选帖子作者标识
- 2层MLP：Linear (x2 拉宽) -> SiLU激活 -> Linear (projection)。
- L2归一化

● Similarity Search：

- 通过dot product，检索Top K的结果。因为用户Emb已经经过L2归一化，等价于cosine

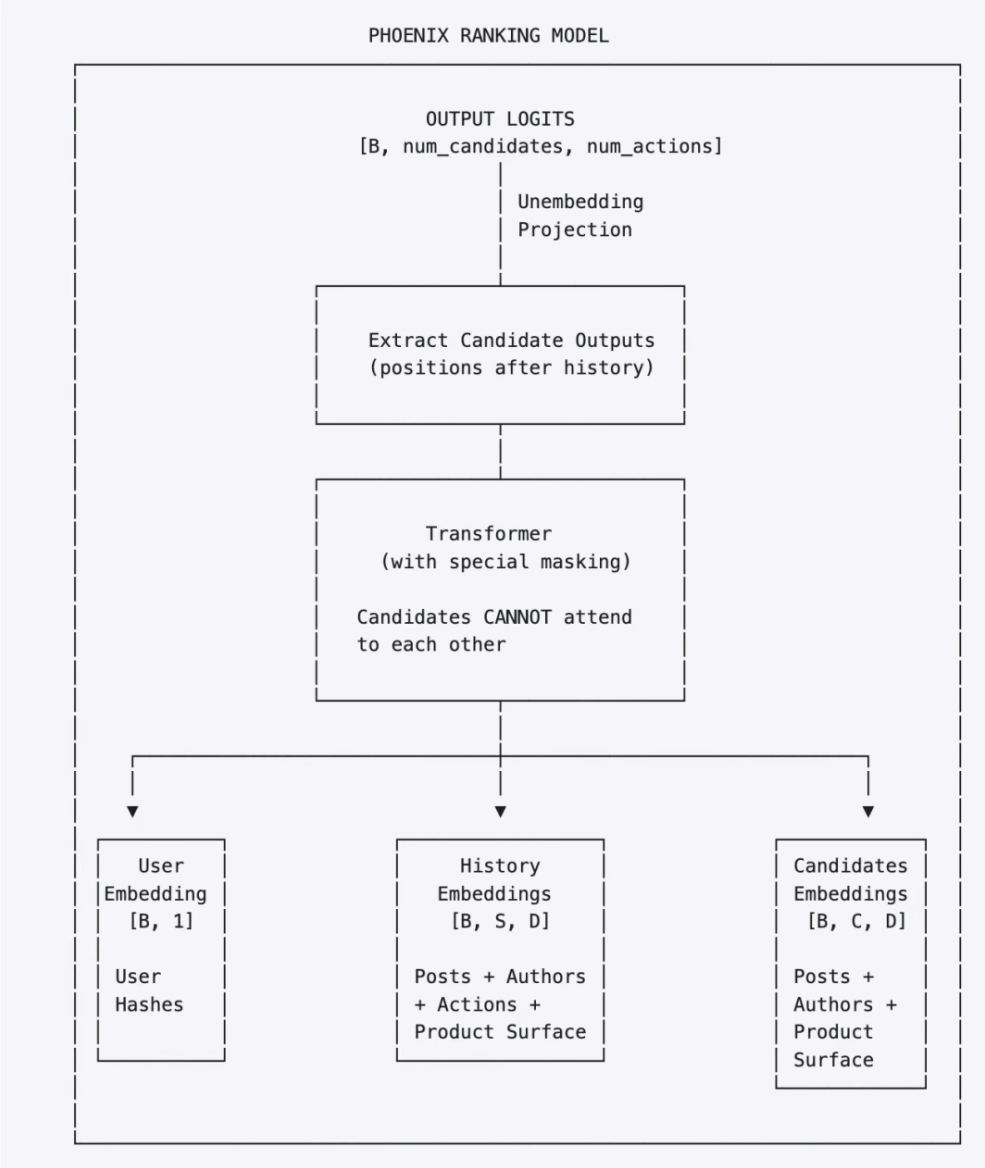
similarity。



- 用户特征：
  - user_hashed：用户唯一标识，通过多个哈希函数映射,num_user_hashes
  - 多个哈希查询的embedding进行拼接，再用可学习的权重矩阵Projection投影到D维
  - 无效的用户padding为0
- 用户历史交互行为，长度为S，每个对象的Sideinfo如下：
  - 历史帖子的TwitterID，每个帖子有多个哈希函数，num_item_hashes
  - 历史作者的AuthorID，每个作者有多个哈希函数，num_author_hashes
  - 历史行为类型，一共19种。是经过投影矩阵得到。
  - 产品界面ID，表达来自哪个界面。通过one-hot单热编码方式查表得到。
  - 先concatenate，序列上每个位置，再用可学习的权重矩阵Projection投影到D维
  - 序列长度不足S的，Padding到S个
- 候选帖子特征：
  - 候选帖子唯一标识，通过多个哈希函数映射，num_item_hashes
  - 候选帖子作者标识，通过多个哈希函数映射，num_author_hashes
  - 先concatenate，再用可学习的权重矩阵Projection投影到D维
  - 候选不足C的，Padding到C个
- 训练精度：BF16+FP32
- 模型参数配置：
  - emb_size = 128

- history_seq_len = 32
- candidate_seq_len = 8
- num_user_hashes = 2
- num_item_hashes = 2
- num_author_hashes = 2
- 产品界面词表大小：16
- FFN扩展因子：2
- 注意力键维度：64
- Query头数量：2
- Key/Value头数量：2
- Transformer层数：2
- 注意力输出缩放系数：0.125

6. Phoenix–模型–排序

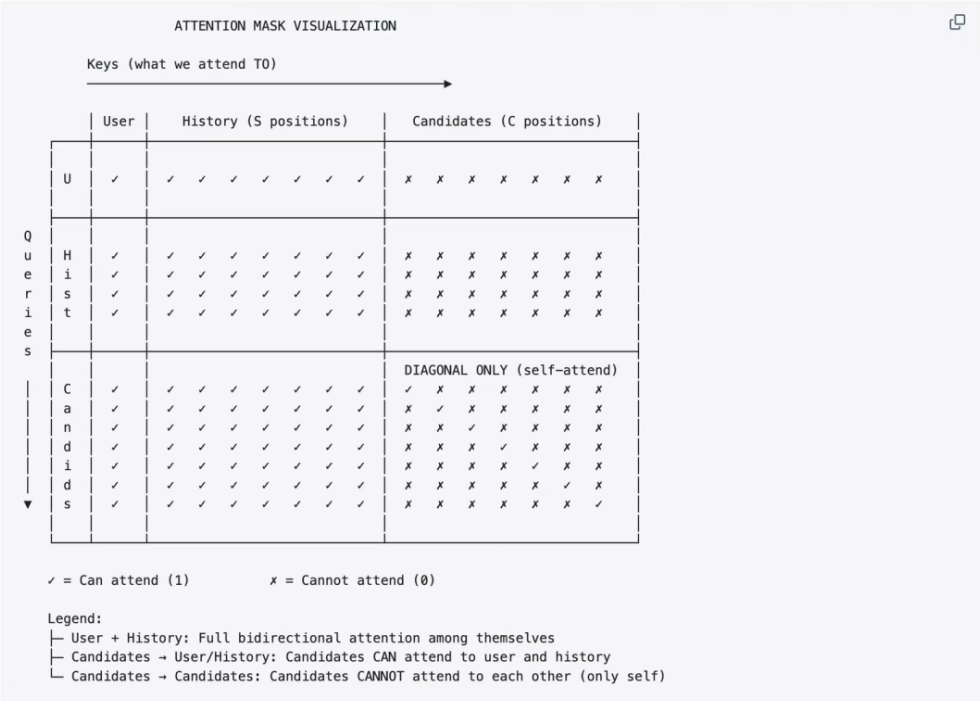


● 用户特征：与Retrieval一致。

● 帖子特征：

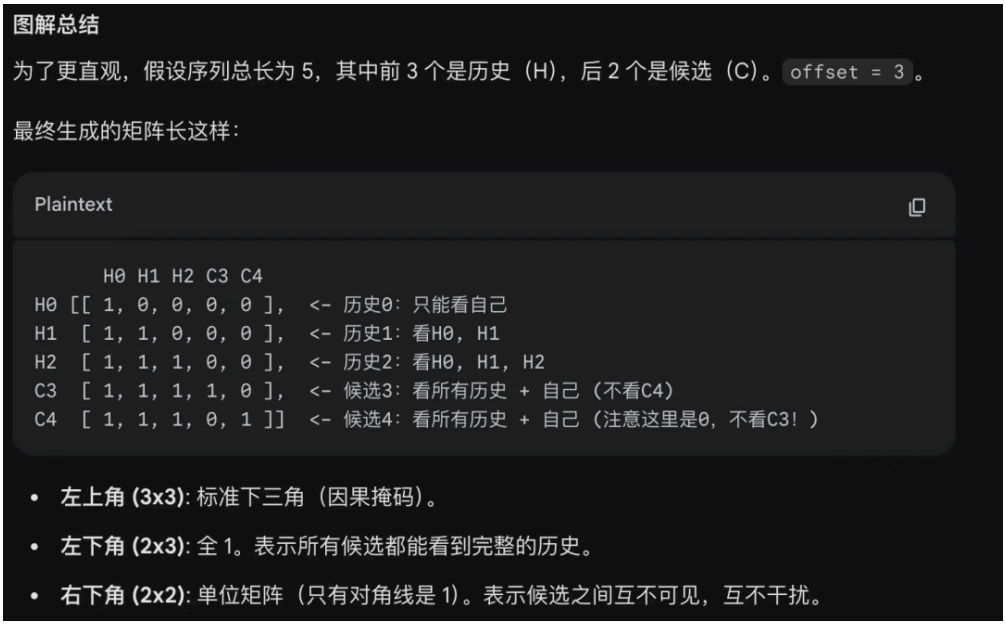
○ 在Retrieval基础上，**多了一个上下文特征，候选帖子将在哪个产品界面展示给用户**。

● **Transformer with Candidate Isolation**（候选隔离的注意力机制），用special attention masking来对候选进行屏蔽，使得候选帖子之间不会互相影响[Candidate Isolation]。



○ 基于代码来看，上述官网的示意图表述不准确，用户与用户行为序列是Causal Attention，非Full Birectional Attention。

○正确版本示意图如下：



○模型输入：用户特征、用户历史行为序列、多个候选帖子

○模型架构：Decoder Only Transformer

■用户特增、用户历史行为：Causal Attention。

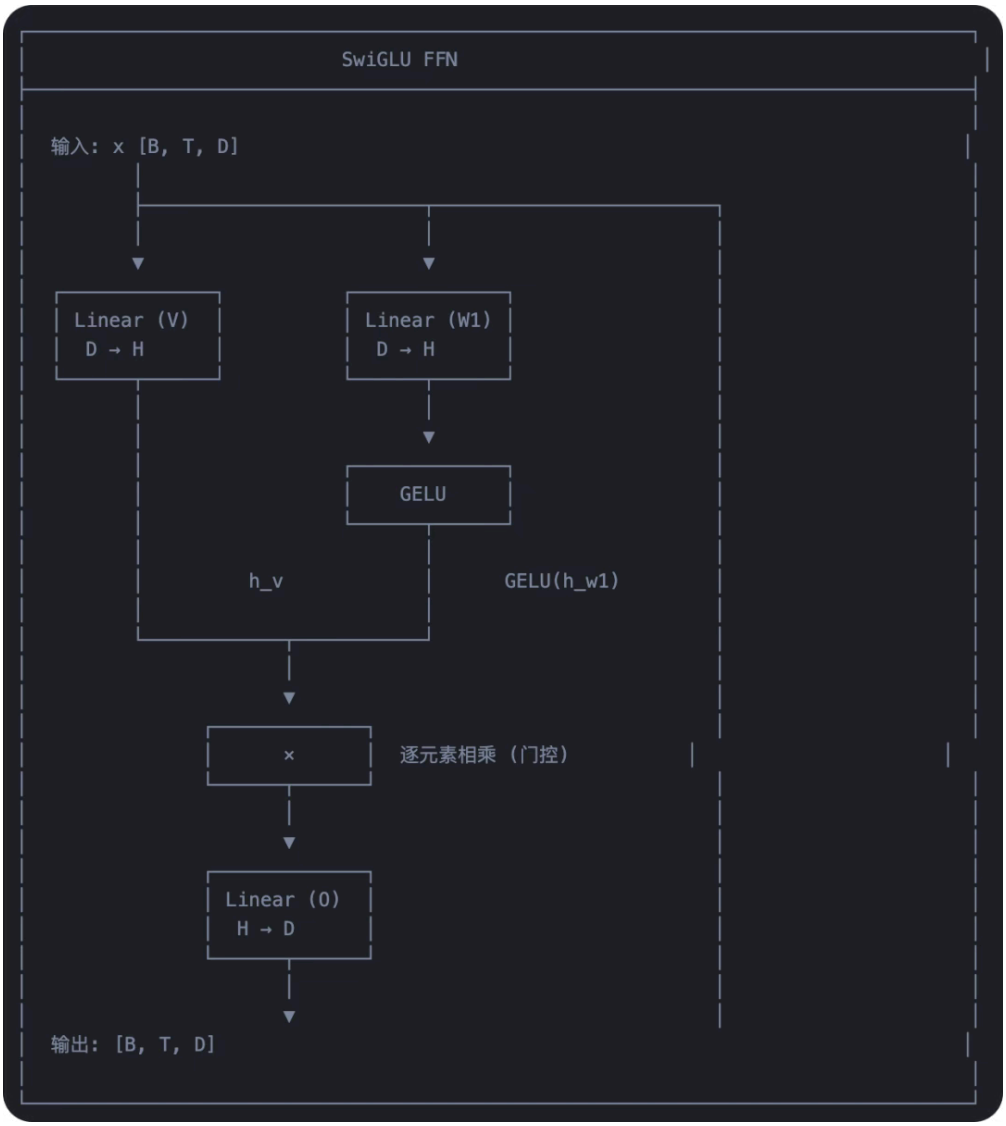
■候选–>用户、用户历史行为、候选：

● 候选可以关注所有用户特征、用户历史行为

● 对候选是只能看到自己，不能看到其他候选

■采用Pre Norm + Post Norm

■FFN采用 SwiGLU FFN： FFN 大小 2.67 × D（因为 SwiGLU 有两条路径）



■抽取用户的history之后的候选位置的输出，该位置表达每个候选的Output。

■乘以[Hidden_Dim, num_actions]形状的权重矩阵，得到每个候选帖子在每种action的原始logit

■Logits 处理：Soft Capping ，用 tanh 将 logits 平滑压缩到 [−M, M]

■sigmoid函数将原始logit转为[0,1]

■**RMSNorm**

■**RoPE位置编码**，RotaryEmbedding

■**Grouped Query Attention**：实现了能力，实际配置KV头数=Q头数。GQA的例子如下：

```
Grouped Query Attention

标准 MHA (num_q_heads = num_kv_heads = 8):
Q: [H1, H2, H3, H4, H5, H6, H7, H8]
K: [H1, H2, H3, H4, H5, H6, H7, H8]
V: [H1, H2, H3, H4, H5, H6, H7, H8]

GQA (num_q_heads = 8, num_kv_heads = 2):
Q: [H1, H2, H3, H4, H5, H6, H7, H8]
K: [    K1    ,     K2    ]  ← 4个Q共享1个K
V: [    V1    ,     V2    ]  ← 4个Q共享1个V

优势: 减少 KV Cache 大小, 加速推理
```

- 输出：
  - [batch_size, num_candidates, 19]
  - 使用其中1个action作为主排序。

```python
ACTIONS: List[str] = [
    "favorite_score",               # 点赞
    "reply_score",                  # 回复
    "repost_score",                 # 转发
    "photo_expand_score",           # 展开图片
    "click_score",                  # 点击
    "profile_click_score",          # 点击个人资料
    "vqv_score",                    # 视频质量观看
    "share_score",                  # 分享
    "share_via_dm_score",           # 通过私信分享
    "share_via_copy_link_score",    # 通过复制链接分享
    "dwell_score",                  # 停留
    "quote_score",                  # 引用转发
    "quoted_click_score",           # 点击引用
    "follow_author_score",          # 关注作者
    "not_interested_score",         # 不感兴趣
    "block_author_score",           # 屏蔽作者
    "mute_author_score",            # 静音作者
    "report_score",                 # 举报
    "dwell_time",                   # 停留时间
]
```

- 模型参数
  - emb_size = 128
  - history_seq_len = 32
  - candidate_seq_len = 8，一次排序的最大候选

- num_user_hashes=2
- num_item_hashes=2
- num_author_hashes=2
- widening_factor=2, # FFN 扩展因子
- key_size=64,# 注意力键维度
- num_q_heads=2,
- num_kv_heads=2,
- num_layers=2, # Transformer 层数
- attn_output_multiplier=0.125, # 注意力输出缩放

● Weighted Scorer，加权线性求和【权重未开源】

- 正向行为：like, repost, share
- 负向行为：block, mute, report

```
Final Score = Σ (weight_i × P(action_i))
```

如对腾讯广告技术感兴趣

可以关注“腾讯广告技术”微信公众号



腾讯广告技术

腾讯广告技术官方阵地，分享团队最新前沿成果及广告技术应用。

230篇原创内容

公众号



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 1.2万

留言

写留言

还没有留言，来写首评